OSSP (Operating Systems And Systems Programming) - 25CS2104E

1. Fork/exec/wait demo — accept a Linux command, run it in a child process, display PIDs of parent and child.

fork_exec.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <string.h>

#define MAX_ARGS 64

int main() {
    char command[256];
    char *args[MAX_ARGS];
    pid_t pid;

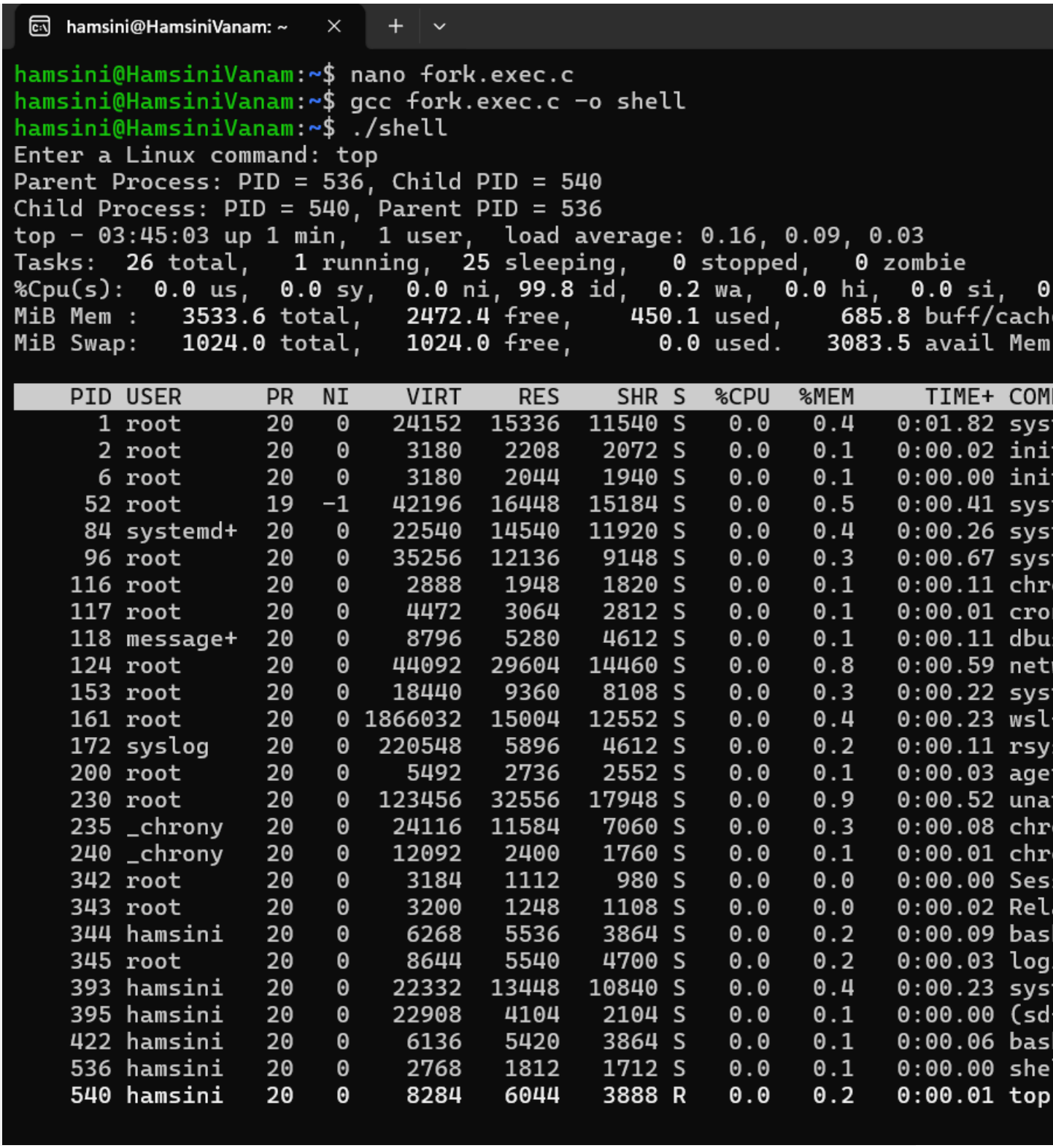
    printf("Enter a Linux command: ");
    fgets(command, sizeof(command), stdin);
    command[strcspn(command, "\n")] = 0;

    int i = 0;
    char *token = strtok(command, " ");
    while (token != NULL) {
        args[i++] = token;
        token = strtok(NULL, " ");
    }
    args[i] = NULL;

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    } else if (pid == 0) {
        printf("Child Process: PID = %d, Parent PID = %d\n", getpid(), getppid());
        execvp(args[0], args);
        perror("exec failed");
        exit(1);
    } else {
        printf("Parent Process: PID = %d, Child PID = %d\n", getpid(), pid);
        wait(NULL);
        printf("Child process completed.\n");
    }

    return 0;
}
```



2. File copy using open()/read()/write()/close(); trace system calls with strace.

copy_file.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

#define BUF_SIZE 1024

int main(int argc, char *argv[]) {
    if (argc != 3) {
        fprintf(stderr, "Usage: %s <source> <destination>\n", argv[0]);
        exit(1);
    }

    int src_fd = open(argv[1], O_RDONLY);
    if (src_fd < 0) {
        perror("open source failed");
        exit(1);
    }

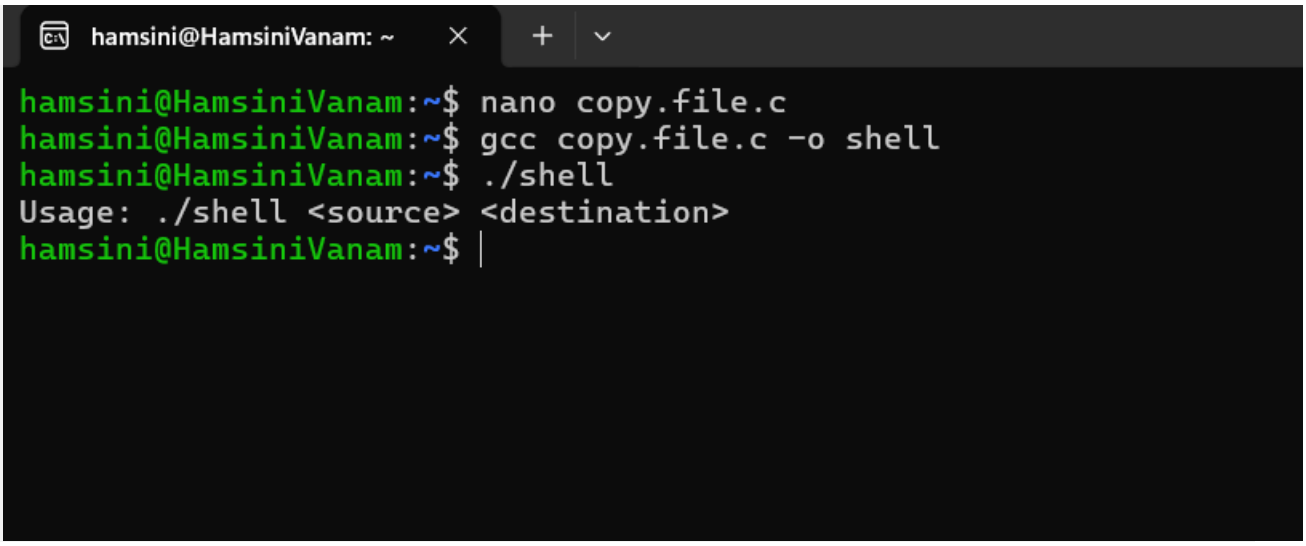
    int dest_fd = open(argv[2], O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (dest_fd < 0) {
        perror("open destination failed");
        close(src_fd);
        exit(1);
    }

    char buffer[BUF_SIZE];
    ssize_t bytes_read;

    while ((bytes_read = read(src_fd, buffer, BUF_SIZE)) > 0) {
        if (write(dest_fd, buffer, bytes_read) != bytes_read) {
            perror("write failed");
            break;
        }
    }

    close(src_fd);
    close(dest_fd);

    printf("File copied successfully.\n");
    return 0;
}
```



3. Fork demo showing PID/PPID and process states; observe state transitions with ps/top//proc.

process_states.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    } else if (pid == 0) {
        printf("Child Process: PID = %d, PPID = %d\n", getpid(), getppid());
        printf("Child state: Running\n");
        sleep(5);
        printf("Child Process: Terminating\n");
        exit(0);
    } else {
        printf("Parent Process: PID = %d, Child PID = %d\n", getpid(), pid);
        printf("Parent state: Waiting for child\n");
        int status;
        waitpid(pid, &status, 0);
        printf("Parent Process: Child terminated, state = Terminated\n");
    }

    return 0;
}
```

```
hamsini@HamsiniVanam:~$ nano process_states.c
hamsini@HamsiniVanam:~$ gcc process_states.c -o shell
hamsini@HamsiniVanam:~$ ./shell
Parent Process: PID = 684, Child PID = 685
Parent state: Waiting for child
Child Process: PID = 685, PPID = 684
Child state: Running
Child Process: Terminating
Parent Process: Child terminated, state = Terminated
hamsini@HamsiniVanam:~$
```

4. Parent creates multiple children, synchronizes with wait() and waitpid(); create and eliminate a zombie process.

multi_child_wait.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define NUM_CHILDREN 3

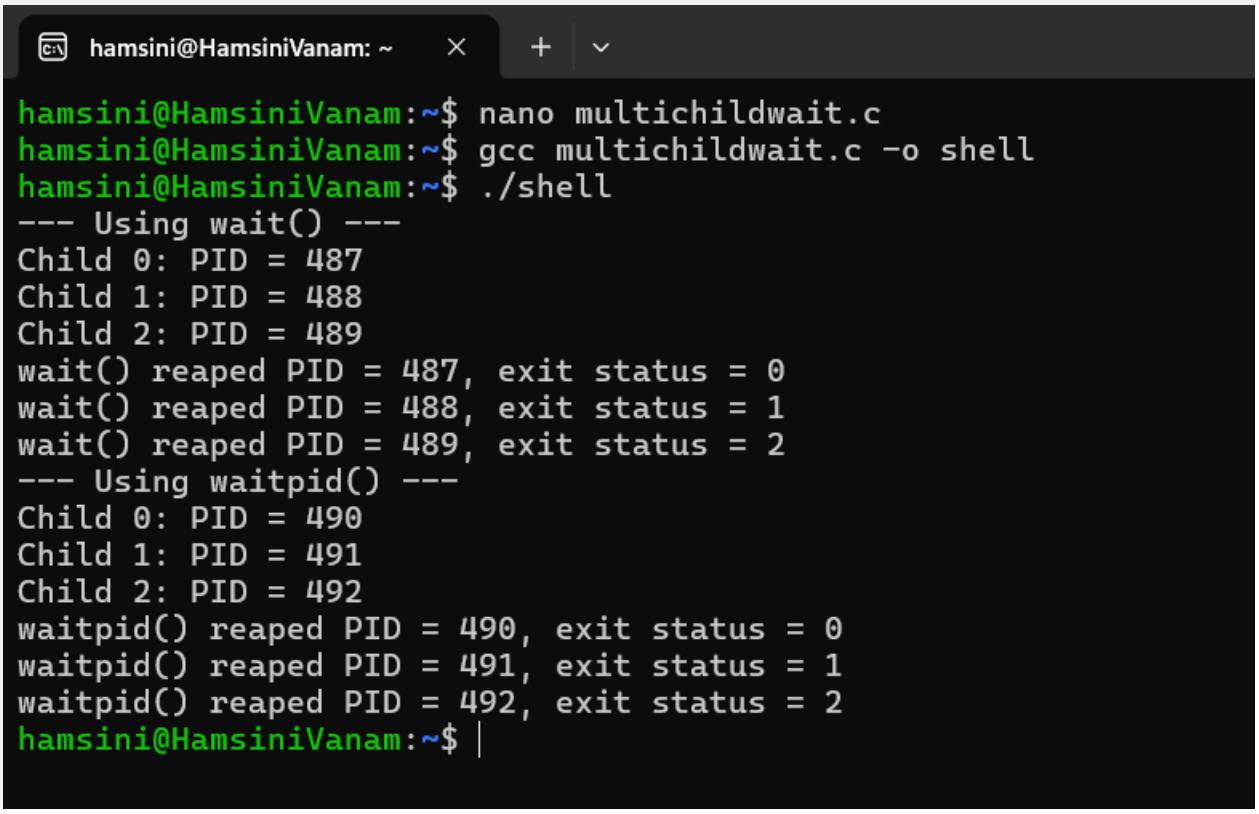
int main() {
    pid_t pids[NUM_CHILDREN];

    printf("--- Using wait() ---\n");
    for (int i = 0; i < NUM_CHILDREN; i++) {
        pid_t pid = fork();
        if (pid == 0) {
            printf("Child %d: PID = %d\n", i, getpid());
            sleep(i + 1);
            exit(i);
        } else {
            pids[i] = pid;
        }
    }
    int status;
    pid_t wpid;
    while ((wpid = wait(&status)) > 0) {
        printf("wait() reaped PID = %d, exit status = %d\n", wpid, WEXITSTATUS(status));
    }

    printf("--- Using waitpid() ---\n");
    for (int i = 0; i < NUM_CHILDREN; i++) {
        pid_t pid = fork();
        if (pid == 0) {
            printf("Child %d: PID = %d\n", i, getpid());
            sleep(i + 1);
            exit(i);
        } else {
            pids[i] = pid;
        }
    }
    for (int i = 0; i < NUM_CHILDREN; i++) {
```

```
        waitpid(pids[i], &status, 0);
        printf("waitpid() reaped PID = %d, exit status = %d\n", pids[i], WEXITSTATUS(status));
    }

    return 0;
}
```



zombie_demo.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid = fork();

    if (pid == 0) {
        printf("Child: PID = %d, exiting now.\n", getpid());
        exit(0);
    } else {
        printf("Parent: PID = %d, child PID = %d\n", getpid(), pid);
        printf("Parent sleeping WITHOUT calling wait() - inspect with: ps -el | grep Z\n");
        sleep(20);

        /* Fix: reap the child to eliminate the zombie */
        wait(NULL);
        printf("Zombie reaped by parent.\n");
    }

    return 0;
}
```

```
hamsini@HamsiniVanam:~$ nano zombie.c
hamsini@HamsiniVanam:~$ gcc zombie.c -o shell
hamsini@HamsiniVanam:~$ ./shell
Parent: PID = 534, child PID = 535
Child: PID = 535, exiting now.
Parent sleeping WITHOUT calling wait() - inspect with: ps -el | grep Z
Zombie reaped by parent.
hamsini@HamsiniVanam:~$
```

5. Producer-consumer over an anonymous pipe; implement 'ls -l | grep ".c"' using fork(), pipe(), dup2(), exec().

pipe_producer_consumer.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>
#include <time.h>

#define MSG_COUNT 5

int main() {
    int fd[2];
    if (pipe(fd) == -1) {
        perror("pipe failed");
        exit(1);
    }

    pid_t pid = fork();

    if (pid == 0) {
        /* Child: consumer */
        close(fd[1]);
        char buffer[100];
        for (int i = 0; i < MSG_COUNT; i++) {
            int n = read(fd[0], buffer, sizeof(buffer) - 1);
            buffer[n] = '\0';
            printf("Consumer received: %s\n", buffer);
        }
        close(fd[0]);
        exit(0);
    } else {
        /* Parent: producer */
        close(fd[0]);
        char msg[100];
        clock_t start = clock();
        for (int i = 0; i < MSG_COUNT; i++) {
            sprintf(msg, "Data item %d", i);
            write(fd[1], msg, strlen(msg));
        }
        close(fd[1]);
        wait(NULL);
        clock_t end = clock();
        printf("Communication time: %f seconds\n", (double)(end - start) / CLOCKS_PER_SEC);
```

```
    }

    return 0;
}
```

```
hamsini@HamsiniVanam:~$ nano pipeprod.c
hamsini@HamsiniVanam:~$ gcc pipeprod.c -o shell
hamsini@HamsiniVanam:~$ ./shell
Consumer received: Data item 0Data item 1Data item 2Data item 3Data item 4
Consumer received:
Consumer received:
Consumer received:
Consumer received:
Communication time: 0.000154 seconds
hamsini@HamsiniVanam:~$
```

ls_grep_pipe.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int fd[2];
    pipe(fd);

    pid_t pid1 = fork();
    if (pid1 == 0) {
        /* First child: runs ls -l, writes to pipe */
        dup2(fd[1], STDOUT_FILENO);
        close(fd[0]);
        close(fd[1]);
        execlp("ls", "ls", "-l", NULL);
        perror("exec ls failed");
        exit(1);
    }

    pid_t pid2 = fork();
    if (pid2 == 0) {
        /* Second child: runs grep .c, reads from pipe */
        dup2(fd[0], STDIN_FILENO);
        close(fd[0]);
        close(fd[1]);
        execlp("grep", "grep", ".c", NULL);
        perror("exec grep failed");
        exit(1);
    }

    close(fd[0]);
    close(fd[1]);
    wait(NULL);
    wait(NULL);

    return 0;
}
```

```
hamsini@HamsiniVanam:~$ nano greppipe.c
hamsini@HamsiniVanam:~$ gcc greppipe.c -o shell
hamsini@HamsiniVanam:~$ ./shell
-rw-r--r-- 1 hamsini hamsini   921 Aug 12 03:46 copy.file.c
-rw-r--r-- 1 hamsini hamsini   104 Aug 11 05:28 fork.c
-rw-r--r-- 1 hamsini hamsini   959 Aug 12 03:44 fork.exec.c
-rw-r--r-- 1 hamsini hamsini   264 Aug 11 05:35 forkexe.c
-rw-r--r-- 1 hamsini hamsini   780 Aug 12 03:58 greppipe.c
-rw-r--r-- 1 hamsini hamsini    78 Aug 11 09:36 hello.c
-rw-r--r-- 1 hamsini hamsini   110 Aug 11 05:26 id.c
-rw-r--r-- 1 hamsini hamsini  1125 Aug 12 03:50 multi_child_wait.c
-rw-r--r-- 1 hamsini hamsini  2250 Aug 12 03:49 multi_child_wait.c.save
-rw-r--r-- 1 hamsini hamsini  1126 Aug 12 03:51 multichildwait.c
-rw-r--r-- 1 hamsini hamsini   230 Aug 11 05:29 parchi.c
-rw-r--r-- 1 hamsini hamsini   288 Aug 11 05:30 pid.c
-rw-r--r-- 1 hamsini hamsini  1074 Aug 12 03:56 pipeprod.c
-rw-r--r-- 1 hamsini hamsini   557 Aug 11 04:03 prac12.c
-rw------- 1 hamsini hamsini   557 Aug 11 04:02 prac12.c.save
-rw-r--r-- 1 hamsini hamsini   532 Aug 11 04:07 prac122.c
-rw-r--r-- 1 hamsini hamsini   713 Aug 12 03:48 process_states.c
lrwxrwxrwx 1 hamsini hamsini     5 Aug 11 09:53 shortcut -> file1
-rw-r--r-- 1 hamsini hamsini   158 Aug 11 05:24 username.c
-rw-r--r-- 1 hamsini hamsini   274 Aug 11 05:31 wait1.c
-rw-r--r-- 1 hamsini hamsini   159 Aug 11 05:32 wait2.c
-rw-r--r-- 1 hamsini hamsini   563 Aug 12 03:53 zombie.c
hamsini@HamsiniVanam:~$
```

6. Client-server using Named Pipes (FIFOs); POSIX signal handling for SIGINT, SIGTERM, SIGUSR1.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define FIFO_NAME "/tmp/myfifo"

int main() {
    char message[256];

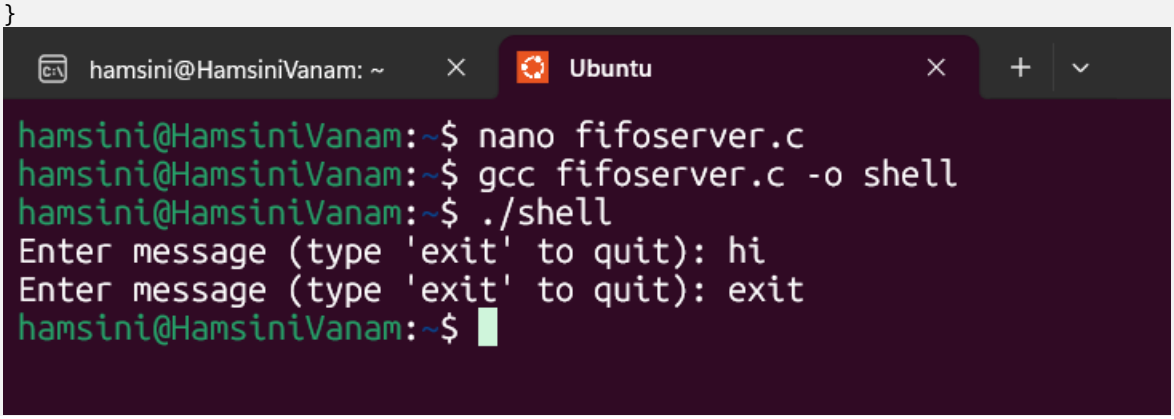
    while (1) {
        printf("Enter message (type 'exit' to quit): ");
        fgets(message, sizeof(message), stdin);
        message[strcspn(message, "\n")] = 0;

        int fd = open(FIFO_NAME, O_WRONLY);
        write(fd, message, strlen(message));
        close(fd);

        if (strcmp(message, "exit") == 0)
```

```
            break;
    }

    return 0;
}
```



signal_handler.c

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void handle_sigint(int sig) {
    printf("\nCaught SIGINT (Ctrl+C). Signal number: %d\n", sig);
}

void handle_sigterm(int sig) {
    printf("\nCaught SIGTERM. Signal number: %d\n", sig);
    exit(0);
}

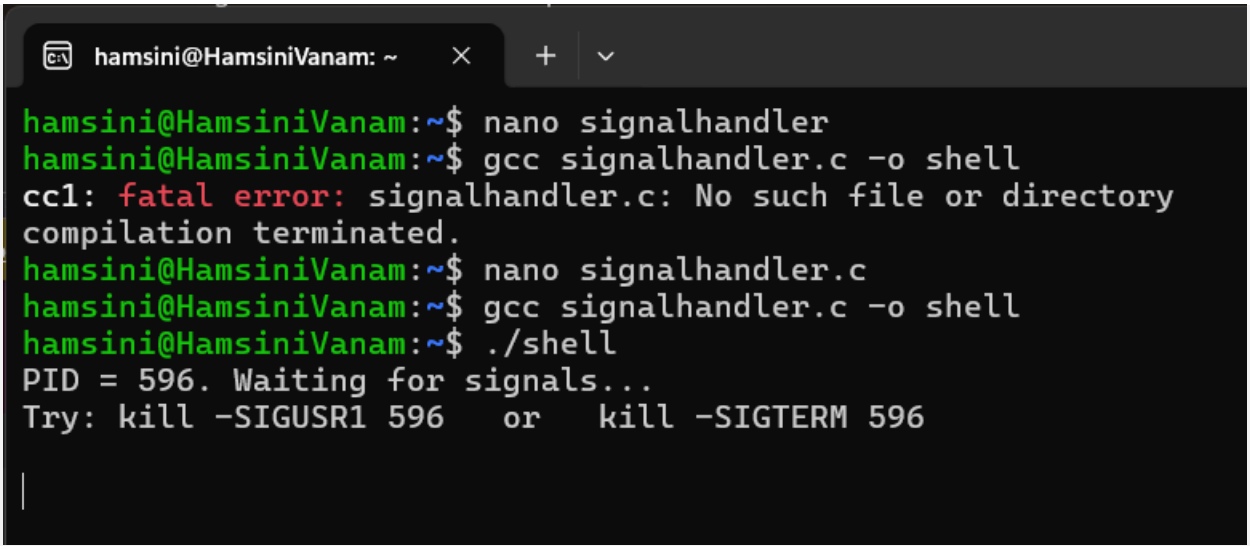
void handle_sigusr1(int sig) {
    printf("\nCaught SIGUSR1. Signal number: %d\n", sig);
}

int main() {
    signal(SIGINT, handle_sigint);
    signal(SIGTERM, handle_sigterm);
    signal(SIGUSR1, handle_sigusr1);

    printf("PID = %d. Waiting for signals...\n", getpid());
    printf("Try: kill -SIGUSR1 %d   or   kill -SIGTERM %d\n", getpid(), getpid());

    while (1) {
        pause();
    }

    return 0;
}
```



7. Print addresses of code, global, static, heap, and stack variables; study /proc/<pid>/maps.

address_space.c

```
#include <stdio.h>
#include <stdlib.h>

int global_var = 10;             /* Data segment */
static int static_var = 20;      /* Data segment */

void print_addresses() {
    int stack_var = 30;                    /* Stack */
    int *heap_var = malloc(sizeof(int));   /* Heap */
    *heap_var = 40;

    printf("Code segment (function addr) : %p\n", (void *)print_addresses);
    printf("Global variable address      : %p\n", (void *)&global_var);
    printf("Static variable address      : %p\n", (void *)&static_var);
    printf("Heap variable address        : %p\n", (void *)heap_var);
    printf("Stack variable address       : %p\n", (void *)&stack_var);

    free(heap_var);
}

int main() {
    print_addresses();
    pause();
    return 0;
}
```

Study virtual memory mappings

```
gcc address_space.c -o address_space
./address_space &
cat /proc/$!/maps
pmap $!
cat /proc/$!/status
```

8. malloc()/calloc()/realloc()/free() with Valgrind leak detection; Copy-on-Write behavior after fork().

mem_alloc.c

```
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *a = malloc(5 * sizeof(int));
    for (int i = 0; i < 5; i++) a[i] = i;

    int *b = calloc(5, sizeof(int));

    a = realloc(a, 10 * sizeof(int));
    for (int i = 5; i < 10; i++) a[i] = i;

    for (int i = 0; i < 10; i++) printf("a[%d] = %d\n", i, a[i]);
    for (int i = 0; i < 5; i++) printf("b[%d] = %d\n", i, b[i]);

    free(a);
    /* 'b' intentionally NOT freed to demonstrate a leak with Valgrind */

    return 0;
}
```

Detect leaks with Valgrind

```
gcc -g mem_alloc.c -o mem_alloc
valgrind --leak-check=full --show-leak-kinds=all ./mem_alloc
```

cow_demo.c

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int data = 100;

int main() {
    printf("Before fork: data = %d, address = %p\n", data, (void *)&data);

    pid_t pid = fork();

    if (pid == 0) {
        printf("Child before write: data = %d, address = %p\n", data, (void *)&data);
        data = 200;   /* Triggers copy-on-write: page is duplicated here */
        printf("Child after write : data = %d, address = %p\n", data, (void *)&data);
    } else {
        wait(NULL);
        printf("Parent after child: data = %d, address = %p\n", data, (void *)&data);
    }

    return 0;
}
```

9. File copy using low-level I/O vs standard library I/O with performance comparison; I/O redirection with dup2().

copy_compare.c

```
#include <stdio.h>
```

```c
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <time.h>

#define BUF_SIZE 4096

void copy_lowlevel(const char *src, const char *dst) {
    int in = open(src, O_RDONLY);
    int out = open(dst, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    char buf[BUF_SIZE];
    ssize_t n;
    while ((n = read(in, buf, BUF_SIZE)) > 0)
        write(out, buf, n);
    close(in);
    close(out);
}

void copy_stdio(const char *src, const char *dst) {
    FILE *in = fopen(src, "r");
    FILE *out = fopen(dst, "w");
    char buf[BUF_SIZE];
    size_t n;
    while ((n = fread(buf, 1, BUF_SIZE, in)) > 0)
        fwrite(buf, 1, n, out);
    fclose(in);
    fclose(out);
}

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <source_file>\n", argv[0]);
        exit(1);
    }

    clock_t t1 = clock();
    copy_lowlevel(argv[1], "copy_lowlevel.out");
    clock_t t2 = clock();
    copy_stdio(argv[1], "copy_stdio.out");
    clock_t t3 = clock();

    printf("Low-level I/O time : %f sec\n", (double)(t2 - t1) / CLOCKS_PER_SEC);
    printf("Standard I/O time  : %f sec\n", (double)(t3 - t2) / CLOCKS_PER_SEC);

    return 0;
}
```

io_redirect.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>

int main() {
    int fd_out = open("output.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
    dup2(fd_out, STDOUT_FILENO);
    close(fd_out);

    printf("This line goes into output.txt, not the terminal.\n");
    fflush(stdout);
```

```
    int fd_in = open("input.txt", O_RDONLY);
    dup2(fd_in, STDIN_FILENO);
    close(fd_in);

    char line[256];
    if (fgets(line, sizeof(line), stdin))
        fprintf(stderr, "Read from input.txt: %s", line);

    return 0;
}
```

10. Investigate inode structures (ls -i, stat, find), hard/symbolic links; file I/O using mmap().

Inode investigation

```
ls -i myfile.txt
stat myfile.txt
find . -inum <inode_number>

ln myfile.txt hardlink.txt
ln -s myfile.txt symlink.txt

ls -li myfile.txt hardlink.txt symlink.txt
stat hardlink.txt
stat symlink.txt
```

mmap_io.c

```
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <ctype.h>
#include <sys/mman.h>
#include <sys/stat.h>

int main(int argc, char *argv[]) {
    if (argc != 2) {
        fprintf(stderr, "Usage: %s <file>\n", argv[0]);
        exit(1);
    }

    int fd = open(argv[1], O_RDWR);
    if (fd < 0) {
        perror("open failed");
        exit(1);
    }

    struct stat sb;
    fstat(fd, &sb);

    char *mapped = mmap(NULL, sb.st_size, PROT_READ | PROT_WRITE, MAP_SHARED, fd, 0);
    if (mapped == MAP_FAILED) {
        perror("mmap failed");
        exit(1);
    }

    printf("File content via mmap:\n%.*s\n", (int)sb.st_size, mapped);
```

```
    if (sb.st_size > 0) {
        mapped[0] = toupper(mapped[0]);
    }

    msync(mapped, sb.st_size, MS_SYNC);
    munmap(mapped, sb.st_size);
    close(fd);

    return 0;
}
```

11. Multithreaded counter using pthreads showing a race condition; fix it with mutex locks.

counter_race.c

```
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 5
#define INCREMENTS 100000

long counter = 0;

void *increment(void *arg) {
    for (int i = 0; i < INCREMENTS; i++) {
        counter++;   /* Not thread-safe: race condition */
    }
    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];

    for (int i = 0; i < NUM_THREADS; i++)
        pthread_create(&threads[i], NULL, increment, NULL);

    for (int i = 0; i < NUM_THREADS; i++)
        pthread_join(threads[i], NULL);

    printf("Final counter value (expected %d) = %ld\n", NUM_THREADS * INCREMENTS, counter);

    return 0;
}
```

Compile & run (repeat a few times to see varying results)

```
gcc -pthread counter_race.c -o counter_race
./counter_race
```

counter_mutex.c

```
#include <stdio.h>
#include <pthread.h>

#define NUM_THREADS 5
#define INCREMENTS 100000

long counter = 0;
pthread_mutex_t lock;
```

```
void *increment(void *arg) {
    for (int i = 0; i < INCREMENTS; i++) {
        pthread_mutex_lock(&lock);
        counter++;
        pthread_mutex_unlock(&lock);
    }
    return NULL;
}

int main() {
    pthread_t threads[NUM_THREADS];
    pthread_mutex_init(&lock, NULL);

    for (int i = 0; i < NUM_THREADS; i++)
        pthread_create(&threads[i], NULL, increment, NULL);

    for (int i = 0; i < NUM_THREADS; i++)
        pthread_join(threads[i], NULL);

    pthread_mutex_destroy(&lock);

    printf("Final counter value (expected %d) = %ld\n", NUM_THREADS * INCREMENTS, counter);

    return 0;
}
```

12. Producer-Consumer using counting semaphores and pthreads; deadlock scenario and prevention via resource ordering.

sem_producer_consumer.c

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>

#define BUFFER_SIZE 5
#define ITEMS 10

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t empty, full;
pthread_mutex_t mutex;

void *producer(void *arg) {
    for (int i = 0; i < ITEMS; i++) {
        sem_wait(&empty);
        pthread_mutex_lock(&mutex);

        buffer[in] = i;
        printf("Produced: %d at index %d\n", i, in);
        in = (in + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&full);
    }
    return NULL;
}
```

```
void *consumer(void *arg) {
    for (int i = 0; i < ITEMS; i++) {
        sem_wait(&full);
        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        printf("Consumed: %d from index %d\n", item, out);
        out = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);
        sem_post(&empty);
    }
    return NULL;
}

int main() {
    pthread_t prod, cons;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
    pthread_mutex_destroy(&mutex);

    return 0;
}
```

deadlock_demo.c

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

pthread_mutex_t lockA = PTHREAD_MUTEX_INITIALIZER;
pthread_mutex_t lockB = PTHREAD_MUTEX_INITIALIZER;

/* --- Deadlock version: opposite lock ordering causes circular wait --- */
void *thread1_deadlock(void *arg) {
    pthread_mutex_lock(&lockA);
    printf("Thread 1 locked A, waiting for B\n");
    sleep(1);
    pthread_mutex_lock(&lockB);
    printf("Thread 1 locked B\n");
    pthread_mutex_unlock(&lockB);
    pthread_mutex_unlock(&lockA);
    return NULL;
}

void *thread2_deadlock(void *arg) {
    pthread_mutex_lock(&lockB);
    printf("Thread 2 locked B, waiting for A\n");
    sleep(1);
    pthread_mutex_lock(&lockA);
```

```
    printf("Thread 2 locked A\n");
    pthread_mutex_unlock(&lockA);
    pthread_mutex_unlock(&lockB);
    return NULL;
}

/* --- Prevention: consistent resource ordering (always A before B) --- */
void *thread1_safe(void *arg) {
    pthread_mutex_lock(&lockA);
    printf("Thread 1 locked A\n");
    sleep(1);
    pthread_mutex_lock(&lockB);
    printf("Thread 1 locked B\n");
    pthread_mutex_unlock(&lockB);
    pthread_mutex_unlock(&lockA);
    return NULL;
}

void *thread2_safe(void *arg) {
    pthread_mutex_lock(&lockA);   /* Same order as thread1: A then B */
    printf("Thread 2 locked A\n");
    sleep(1);
    pthread_mutex_lock(&lockB);
    printf("Thread 2 locked B\n");
    pthread_mutex_unlock(&lockB);
    pthread_mutex_unlock(&lockA);
    return NULL;
}

int main() {
    pthread_t t1, t2;

    /* Reproduce the deadlock: swap thread1_safe/thread2_safe for
       thread1_deadlock/thread2_deadlock below. Program will hang;
       inspect with: ps -eLo pid,tid,stat,wchan:32,comm */
    printf("--- Deadlock-safe version (consistent resource ordering) ---\n");
    pthread_create(&t1, NULL, thread1_safe, NULL);
    pthread_create(&t2, NULL, thread2_safe, NULL);
    pthread_join(t1, NULL);
    pthread_join(t2, NULL);

    return 0;
}
```